Estado	Finalizado
Comenzado	miércoles, 7 de octubre de 2026, 18:40
Completado	miércoles, 7 de octubre de 2026, 19:14
Duración	33 mins 57 segs

Pregunta 1

Finalizado

Sin calificar

Explicá qué son las cabeceras HTTP y para qué sirven. Describí al menos cuatro cabeceras de petición y explicá la diferencia entre `Content-Type` y `Accept`.

Las cabeceras HTTP transportan metadatos esenciales sobre la petición, la respuesta, el cliente y el servidor. Se escriben como pares clave-valor separados por dos puntos (Clave: Valor), cada uno en una línea individual.

Para qué sirven: definen reglas de negociación de formato, autenticación, gestión de caché y control de seguridad.

Cabeceras de petición

Son la información contextual enviada desde el cliente hacia el servidor:

- Host: api.ejemplo.com: nombre del dominio de destino. Es obligatoria en HTTP/1.1.
- User-Agent: Mozilla/5.0...: identifica el navegador, el motor y el sistema operativo del cliente.
- Accept: application/json: informa qué tipo de formato espera recibir de respuesta.
- Authorization: Bearer token...: credenciales de seguridad para autenticar la solicitud.

Diferencia entre Content-Type y Accept

Content-Type especifica el formato exacto del payload adjunto, mientras que Accept informa qué tipo de formato espera recibir de respuesta; es decir, Content-Type dice "el contenido que te mando está en este formato", y Accept dice "quiero que me respondas en este formato".

Las cabeceras son pares Clave: Valor que configuran la sesión, la seguridad y el tipo de contenido que viaja. Son metadatos del mensaje, no su contenido, y existen tanto en peticiones como en respuestas. Gracias a ellas, quien recibe el mensaje sabe cómo tratarlo sin adivinar. Son como el sobre de una carta, mientras que el cuerpo es la carta.

Algunas cabeceras de petición son:

- **Host** (`api.midominio.com`): especifica el dominio del servidor al que se envía la petición.
- **Authorization** (`Bearer <TOKEN_JWT>`): pasa las credenciales de acceso para demostrar quién es el usuario.
- **Content-Type** (`application/json`): avisa al servidor en qué formato vienen los datos enviados en el payload.
- **Accept** (`application/json`): le dice al servidor en qué formato prefiere el cliente recibir la respuesta.
- **User-Agent** (`Mozilla/5.0…`): identifica el dispositivo, el sistema operativo y el navegador que hace la consulta.

La diferencia es que `Content-Type` describe lo que el cliente envía y `Accept` describe lo que el cliente quiere recibir.

Pregunta 2

Finalizado

Sin calificar

Explicá qué hace cada una de estas cabeceras de respuesta: `Content-Type`, `Set-Cookie`, `Access-Control-Allow-Origin` y `Cache-Control`. Incluí un ejemplo de valor en cada caso.

- Content-Type: Especifica el formato exacto del payload adjunto. Ej. Content-Type: application/json
- Set-Cookie: Ordena al navegador guardar una cookie de sesión. Ej. Set-Cookie: session=xyz...
- Access-Control-Allow-Origin Reglas de seguridad CORS para peticiones entre dominios.
- Cache-Control: Instrucciones sobre cuánto tiempo guardar en caché local. Ej. Cache-Control: max-age=3600

- **Content-Type** (`application/json; charset=utf-8`): informa al cliente que los datos del payload están formateados en JSON y con qué codificación de caracteres. El navegador decide cómo tratar el contenido según este valor.
- **Set-Cookie** (`session_id=xyz123; Secure; HttpOnly`): le pide al navegador que guarde una cookie para recordar la sesión activa. Los atributos `Secure` y `HttpOnly` agregan protección.
- **Access-Control-Allow-Origin** (`*`): es parte de CORS e indica qué dominios externos tienen permiso para consumir esa API desde un navegador. El `*` significa que cualquier dominio puede hacerlo.
- **Cache-Control** (`max-age=3600`): instruye al navegador a guardar el resultado en memoria local durante 1 hora (3600 segundos) antes de volver a pedirlo. Si se repite la petición dentro de ese tiempo, el navegador usa su copia sin consultar al servidor.

Pregunta 3

Finalizado

Sin calificar

Explicá cómo se organizan los códigos de estado HTTP en familias. Luego indicá qué código esperarías y por qué en estos casos: (a) un POST que crea un usuario con éxito, (b) un usuario con sesión iniciada que intenta entrar a una sección de administradores, (c) registrar un usuario con un email que ya existe, (d) un servidor en mantenimiento.

Un código de estado es un número de 3 dígitos que indica el resultado de la petición, y el primer dígito define la familia:

1xx: Informativo.

2xx: Éxito (todo salió bien).

3xx: Redirección (el recurso se movió).

4xx: Error del cliente (te equivocaste vos).

5xx: Error del servidor (se rompió el servidor backend).

(a) un POST que crea un usuario con éxito: 201 Created

(b) un usuario con sesión iniciada que intenta entrar a una sección de administradores: 403 Forbidden

(c) registrar un usuario con un email que ya existe: 401 Unauthorized

(d) un servidor en mantenimiento: 503 Service Unavailable

Los códigos de estado son la respuesta rápida del servidor para indicar qué sucedió. Se dividen en cinco familias según su primer dígito:

- **1xx:** informativos, la petición fue recibida y el proceso continúa.
- **2xx:** éxito, la acción fue recibida, entendida y aceptada.
- **3xx:** redirección, el cliente debe hacer algo más para completar la solicitud.
- **4xx:** errores del cliente, el problema está en quien envió la petición.
- **5xx:** errores del servidor, la petición era correcta pero el servidor falló al procesarla.

Casos:

- **(a) 201 Created:** la petición tuvo éxito y se creó un nuevo recurso, típico tras un POST.
- **(b) 403 Forbidden:** el usuario se autenticó correctamente pero no tiene permisos. No corresponde 401, que se usa cuando faltan credenciales de autenticación.
- **(c) 409 Conflict:** la petición choca con el estado actual del sistema, porque el email ya existe.
- **(d) 503 Service Unavailable:** el servidor está saturado o en mantenimiento temporal.

Pregunta 4

Finalizado

Sin calificar

Explicá qué problema tiene HTTP en cuanto a seguridad y cómo lo resuelve HTTPS. Describí qué aporta TLS y qué significa realmente el candado del navegador.

HTTP (puerto 80) transmite en texto plano, legible por cualquier nodo intermedio de la red. Por eso es vulnerable a ataques y a la captura de contraseñas.

HTTPS (puerto 443) es la versión segura de HTTP, respaldada por la capa de cifrado TLS/SSL. Ofrece cifrado de extremo a extremo mediante algoritmos criptográficos, y con eso garantiza confidencialidad, autenticidad e integridad de datos

TLS es la capa de seguridad que cifra la información enviada y convierte a HTTP en HTTPS. Actúa en la etapa 2 del ciclo de una petición, cuando se construye la "tubería" cifrada antes de transmitir ningún dato. Allí se hace el handshake: el cliente pregunta si el servidor está listo para hablar de forma segura, el servidor responde con sus claves de cifrado y su certificado digital, y el cliente confirma que la conexión segura quedó establecida.

Usa un cifrado híbrido:

Cifrado asimétrico: clave pública y privada para negociar la sesión inicial de forma segura.

Cifrado simétrico: una clave única de sesión, ultrarrápida, para cifrar el tráfico de datos.

Los certificados digitales son emitidos por Autoridades de Certificación (CA) y validan la identidad.

El candado del navegador confirma que el certificado fue emitido por una CA válida y que las claves criptográficas están activas. Es decir, indica que la conexión está cifrada y que la identidad del servidor fue validada por una autoridad de certificación.

HTTP viaja en texto plano, por lo que cualquiera que pueda observar el camino de los datos, por ejemplo en un Wi-Fi público, puede leer contraseñas y datos. Además, el contenido puede alterarse en tránsito y es posible suplantar al servidor.

HTTPS es HTTP más TLS: la misma comunicación, pero dentro de una conexión cifrada. TLS aporta:

- **Cifrado (confidencialidad):** lo que viaja no puede leerse en el camino.
- **Integridad:** se puede detectar si el contenido fue alterado.
- **Autenticidad:** el servidor presenta un certificado digital que permite verificar que es quien dice ser.

Esto se establece en el handshake de la etapa TCP/TLS: el cliente pregunta si pueden comunicarse de forma segura, el servidor envía su certificado y claves, y se acuerda la clave de sesión.

El candado indica que la conexión está cifrada y que el dominio fue verificado. No garantiza que el sitio sea honesto.

Puntos clave: identifica los riesgos de HTTP, nombra las tres garantías de HTTPS y aclara el alcance del candado.

Pregunta 5

Finalizado

Sin calificar

HTTP es un protocolo sin estado. Explicá qué significa esto y qué problema genera. Explicá cómo las cookies resuelven ese problema, indicando qué cabeceras intervienen y qué función cumplen `Secure` y `HttpOnly`.

HTTP es stateless: no recuerda nada entre una petición y la siguiente. Esa es la razón por la que las cookies permiten "recordar sesiones de usuario".

Gracias a las cookies podés navegar entre 50 productos de un e-commerce sin volver a ingresar tus credenciales en cada clic. El mecanismo usa dos cabeceras:

- Set-Cookie (guardado): el servidor la envía en la respuesta para almacenar un identificador. Ejemplo: Set-Cookie: session_id=abc123. Ordena al navegador guardar una cookie de sesión.
- Cookie (envío): el navegador reenvía automáticamente esta cabecera en cada petición subsiguiente. Ejemplo: Cookie: session_id=abc123.

Atributos de Seguridad de las Cookies

Flags de configuración fundamentales para proteger los tokens de sesión frente a ataques.

- HttpOnly Impide que scripts de JavaScript (document.cookie) accedan a la cookie. Mitiga ataques XSS.
- Secure Instruye al navegador a transmitir la cookie únicamente a través de conexiones cifradas HTTPS.
- SameSite Restringe el envío de la cookie en peticiones entre distintos sitios. Mitiga ataques CSRF.

Que HTTP sea sin estado significa que no recuerda las peticiones anteriores: cada una es independiente. El problema es que el servidor no podría saber, por ejemplo, que un usuario ya inició sesión.

Las cookies lo resuelven así:

1. El servidor entrega un identificador de sesión mediante la cabecera de respuesta **Set-Cookie**.
2. El navegador lo guarda.
3. En cada petición posterior al mismo sitio, el navegador lo reenvía en la cabecera de petición **Cookie**, y así el servidor reconoce al usuario.

Atributos de seguridad:

- **Secure:** la cookie solo se envía por HTTPS.
- **HttpOnly:** JavaScript no puede leerla.

Normalmente la cookie guarda un identificador y no la contraseña. Es como la pulsera de un parque: se paga una vez y luego solo se muestra en cada juego.